

Cloud Computing and Virtualization

Bachelors in Computer Engineering

**Prepared by Er. Ganga Gautam,
Assistant Professor, Pokhara University**

Chapter 3

Storage in Cloud computing(5 hrs)

Outlines:

1. Database as a Service
2. Storage model and file system
 - 2.1. Parallel file system and Distributed file system
3. Cloud Storage Services
 - 3.1. Object storage, file storage and block storage, Archive Storage
 - 3.2. Elastic storage, Data redundancy
4. Databases
 - 4.1. Relational: SQL, Non-relational: NoSQL and Graph Database

Do you know Database??

Importance of Storage in Cloud

- It is the foundation of all cloud services — whether SaaS, PaaS, or IaaS.
- Every layer of cloud computing depends on storage.
 - In SaaS, storage hosts application data.
 - In PaaS, it supports development platforms.
 - In IaaS, it provides virtual disks and object repositories.
- Enables global accessibility and data sharing.
- Reduces infrastructure costs and administrative overhead.

Measures	On-Premises Database	Cloud Database / DBaaS
Reliability	Reliable, and Private.	More reliable but not necessarily Private.
Scalability	Limited scalable.	Unlimitedly scalable.
Speed	Faster, but may fail in any point of time (in care of hardware failure).	Faster and will be up always.
Deployment	Deployment takes time.	Deployed within no time.
Cost Effectiveness	Lots of capital required to setup on-premises database as a service.	Pay only for what you Use. Highly Cost effective. No Overhead cost involved.
Maintenance	High on Maintenance Cost. All cost to the company. HW, technicians, DBA's and other infrastructure.	No Maintenance Cost. Pay for what you use.
Setup Cost	Entire Setup cost is to be borne by the Company.	Entire Setup cost is borne by Vendor. The company pays only for the Service.
Security	Highly Secure and Controlled.	Highly Secured as per Vendor.

Cloud Storage

- On-demand data storage service
- Allows users to save, access, and manage data via the internet instead of local drives.
- Offers high reliability, scalability, and security compared to traditional on-premises storage.
- It is a core component of cloud computing, storing the data that applications use.
- Supports key applications, including:
 - Big Data Analytics
 - Data Warehouses
 - Internet of Things (IoT)
 - Databases
 - Backup and Archive Solutions
 - Best for: Storing unstructured data (files, media).

- Cloud storage is used to store files and objects, such as:
 - Images
 - Videos
 - PDFs
 - Backups
 - Large data files
- It works like Google Drive or OneDrive but on a larger, scalable scale.
- Provides flexible data storage architectures for modern applications.
- **Examples:**
 - Amazon S3
 - Google Cloud Storage
 - Azure Blob Storage

Core Features:

- **Elasticity**
 - Capacity can grow or shrink automatically.
- **Scalability**
 - Handle petabytes of data with ease.
- **Redundancy**
 - Multiple copies of data stored across regions for reliability.
- **Managed Infrastructure**
 - Provider handles hardware, updates, and backups.

Database as a Service (DBaaS)

- A cloud-based model where a third-party provider hosts, manages, and maintains the database.
- Provides on-demand access to a managed database in the cloud, without requiring physical setup or maintenance.
- Users access the database via the cloud without managing internal setup.
- Pricing usually follows a subscription or usage-based model.
- Lets organizations focus on applications and development, not database administration.
- **Examples:**
 - AWS RDS (Relational Database Service)
 - Azure SQL Database
 - Google Cloud SQL / Firestore

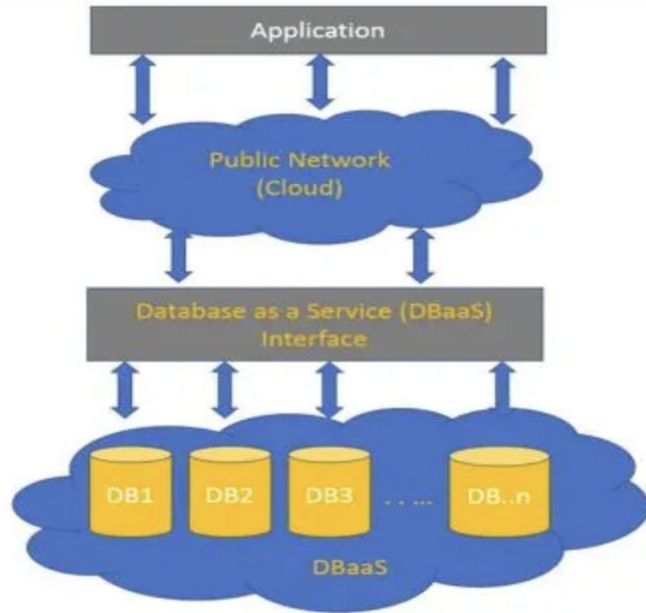
Example: MongoDB's DBaaS — MongoDB Atlas

- MongoDB Atlas is the DBaaS offering by MongoDB.
- It provides globally distributed database deployment across many regions — useful for multi-region apps or services needing data locality and low latency.
- Atlas offers security features like end-to-end encryption, always-on authentication, automated backups, and hosted infrastructure management — giving you enterprise-level database features without the overhead.

Characteristics of DBaaS

- Fully managed by the cloud provider.
- No manual installation, configuration, or patching.
- Security & performance management
- Supports multiple database types:
 - SQL (Relational) – MySQL, PostgreSQL, Oracle.
 - NoSQL (Non-relational) – MongoDB, DynamoDB.
 - NewSQL – Combines SQL structure with NoSQL scalability.

DBaaS Architecture



Cloudbase Database Architecture (DBaaS)

Advantages of DBaaS

- **Cost Efficiency:**
 - Pay-as-you-go pricing, no hardware costs.
- **Scalability:**
 - Easily increase or decrease resources.
- **Reliability:**
 - Automatic failover and backup options.
- **Security:**
 - Encryption, access control, and compliance.
- **Time-Saving:**
 - No manual updates, patching, or monitoring required.
- **Integration:**
 - Works well with analytics, AI, and data processing services.
- **Reduced operational overhead**
 - provider handles updates, patches, backups, scaling, and security

Disadvantages / Challenges

- **Vendor Lock-in**
 - Migration between providers can be complex.
- **Latency Issues**
 - Data access delays if servers are in different regions.
- **Limited Customization**
 - Users cannot tweak low-level configurations.
- **Compliance Challenges**
 - Must meet regional data regulations (e.g., GDPR).
- **Dependency on Internet**
 - Downtime = no access to the database.

Case Study: Netflix using AWS RDS + DynamoDB

Example:

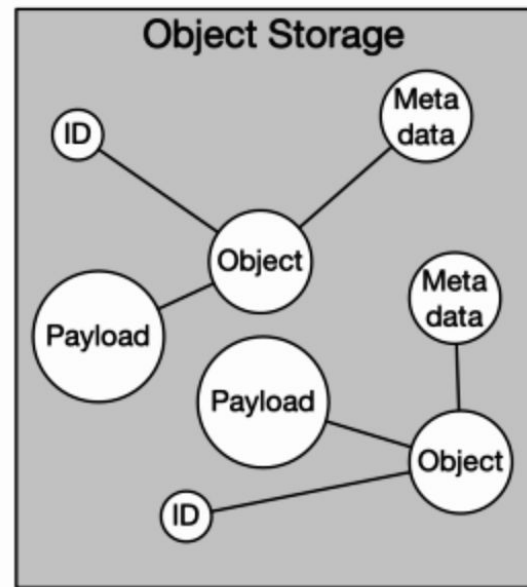
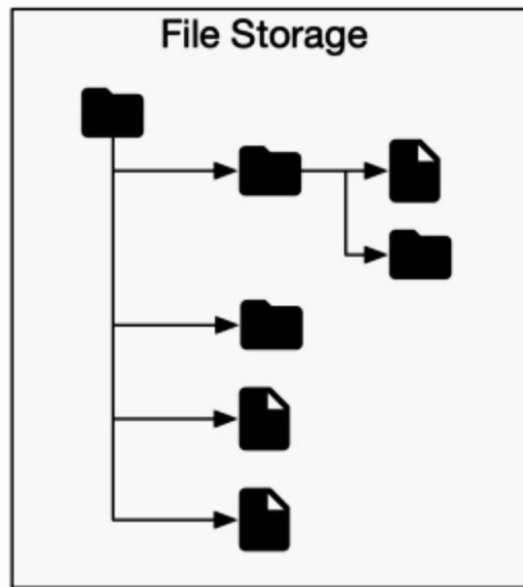
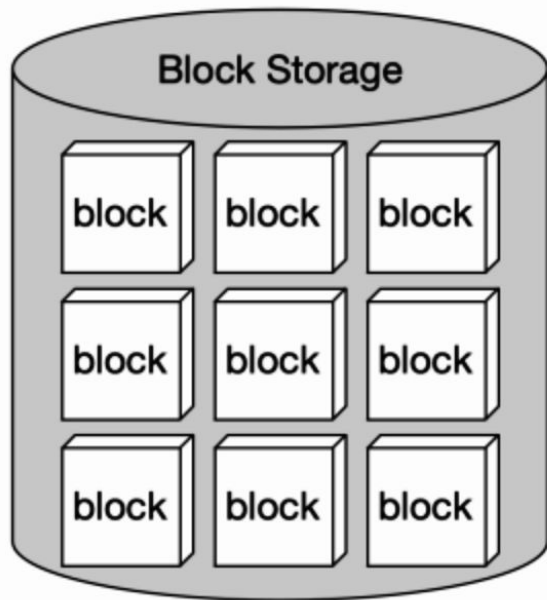
- Netflix, one of the largest streaming platforms, relies heavily on AWS cloud storage and databases.
- Uses:
 - AWS RDS (Relational DB): For user account and transactional data.
 - Amazon DynamoDB (NoSQL): For session data and fast lookups.
 - S3 (Object Storage): For video content and backups.

Outcome:

- High availability, low latency, auto-scaling databases, and global distribution.

Storage Model

- Cloud storage models define how data is stored, managed, accessed, and organized in cloud environments.
- Cloud platforms (AWS, Azure, GCP) offer multiple storage models, each designed for different performance, access patterns, and cost requirements.
- Major Cloud Storage Models
 - Object Storage
 - Block Storage
 - File Storage
- Each model serves different performance, structure, and scalability needs.
- Each model operates at a different abstraction level.
 - For example, object storage is accessed via HTTP APIs, whereas file and block operate closer to the OS.
- Cloud providers often integrate all three models to cater to diverse workloads.



Object Storage

- Object storage stores data as objects, which contain:
 - The data itself
 - Metadata (details about the data)
 - Unique ID
- Highly Scalable Cloud Storage
- **Key Features:**
 - Stores unstructured data (images, video, logs, backups, datasets)
 - Accessed using REST APIs (HTTP/HTTPS)
 - Unlimited scalability
 - Objects cannot be partially updated; must rewrite whole object
 - Highly durable (11–12 nines)

- **Structure of an Object:**
 - Object Data – actual content
 - Metadata – size, type, creation date, custom tags
 - Unique Identifier (ID) – used to locate object
- **Use Cases**
 - Backups, archives
 - Multimedia storage (images, videos, audio)
 - Logs, analytics data
 - Static website hosting
- **Examples**
 - Amazon S3, Google Cloud Storage, Azure Blob Storage

Advantages

- Massive scalability
- Cheap storage cost
- High durability across multiple regions
- Easy access over the internet (URL-based)
- Ideal for backups and content distribution

Limitations

- Slower performance compared to block storage
- Not suitable for databases or OS files
- No directory hierarchy (uses buckets instead)

Block Storage

- Stores data in fixed-size blocks, similar to a traditional hard drive.
- High Performance Storage
- **Key Features:**
 - High performance, low latency
 - Can be attached to a VM as a drive
 - Suitable for transactional workloads
 - Allows partial updates (modify a single block)

Advantages

- Very high IOPS & speed
- Good for databases and OS disks
- Supports RAID, encryption, snapshots
- Predictable performance

Limitations

- Limited to the size of the attached volume
- Not internet-accessible
- More expensive than object storage
- Requires full management of file system

- **Use Cases**

- Databases (MySQL, PostgreSQL)
- Virtual machines
- Boot volumes
- Enterprise applications requiring low latency

- **Examples**

- Amazon EBS, Azure Disk Storage, Google Persistent Disk

File Storage

- File Storage organizes data in folders and directories, similar to traditional file systems.
- Provides a hierarchical file and folder structure accessible over the network.
- Shared File System in Cloud
- **Key Features:**
 - Uses NFS, SMB, AFP protocols
 - Shared file system accessible by multiple VMs/servers
 - Supports file locking and permissions
 - Easy migration for existing on-premise workloads

Advantages

- Familiar file system structure (folders, directories)
- Multi-user and multi-application access
- Supports shared workloads (e.g., home directories)
- Simple integration with enterprise systems

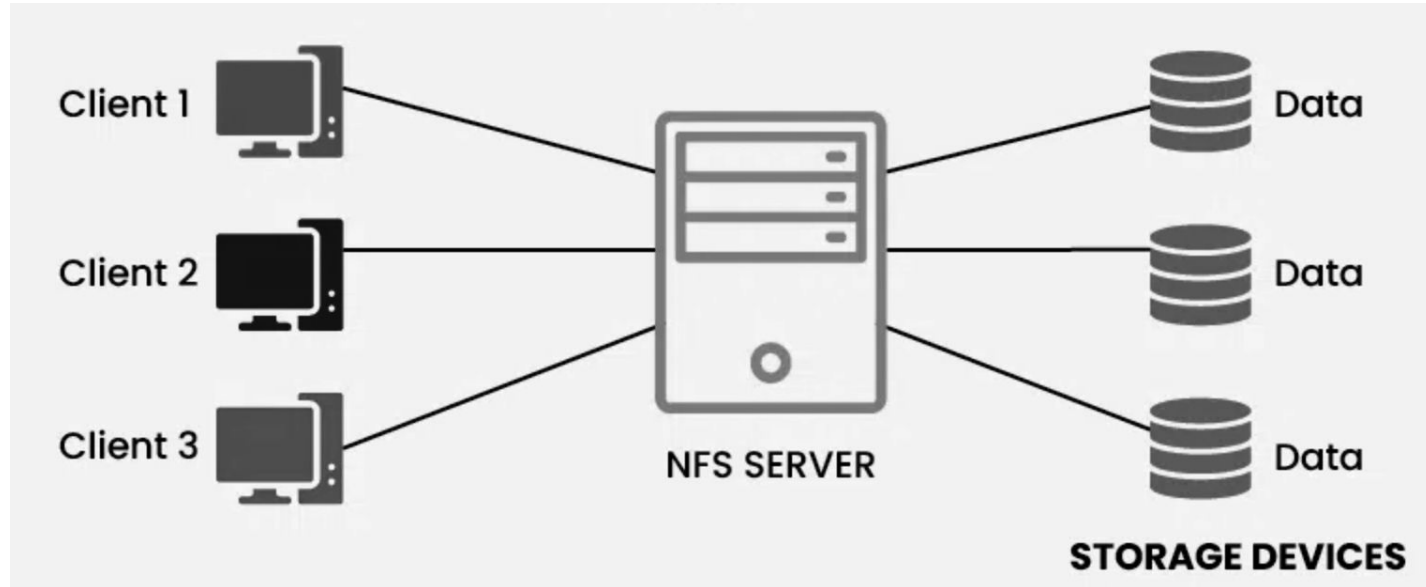
Limitations

- Limited scalability compared to object storage
- Higher cost than block storage
- Performance lower for large databases

- **Use Cases**
 - Shared file repositories
 - Content management
 - Development environments
 - Home directories for users
 - Media editing/workflows
- **Examples**
 - Amazon EFS, Azure Files, Google Filestore

Distributed File System

- A Distributed File System stores and manages files across multiple networked servers, making them appear as a single unified file system to users.



- **Key Characteristics**

- Location Transparency:
 - Users don't need to know where the file is physically stored.
- Scalability:
 - New storage nodes can be added easily.
- Fault Tolerance:
 - Replication ensures data is available even if a node fails.
- Distributed Access:
 - Multiple clients can access files across the network.
- Load Balancing:
 - Workload is distributed across nodes to improve performance.

How DFS Works

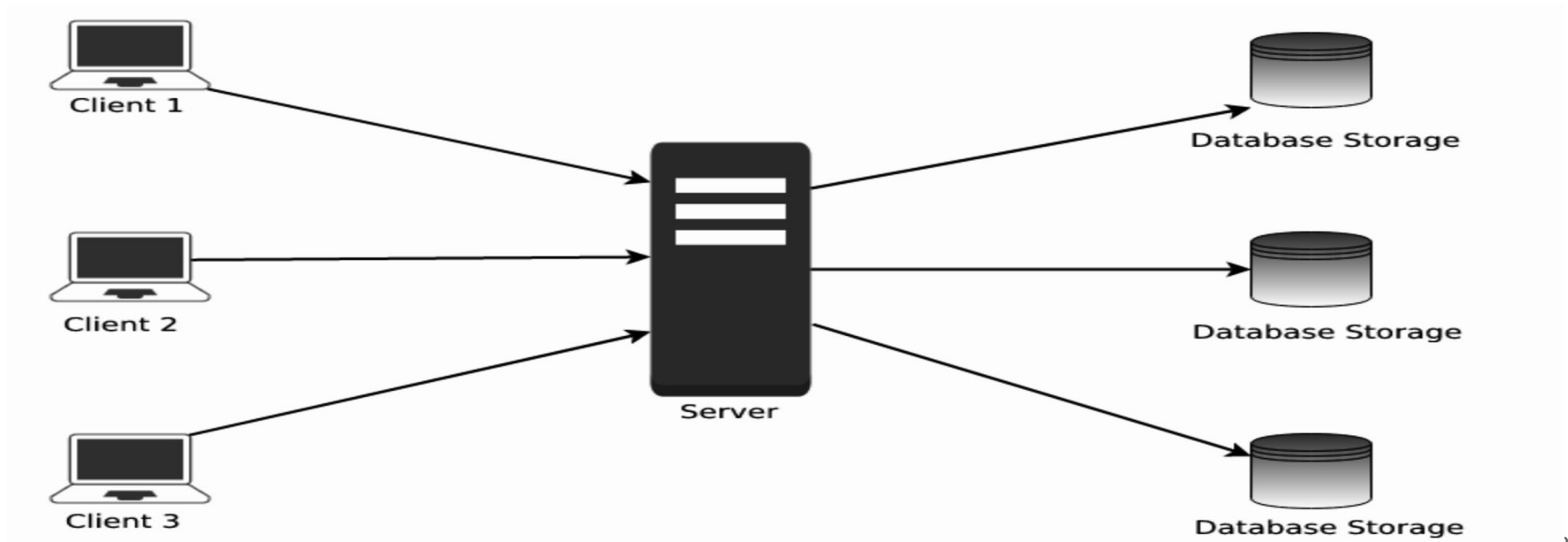
- Files are divided and stored across several servers.
- Metadata servers manage file locations.
- Clients communicate with the system using a DFS protocol (e.g., NFS, SMB).
- Replication keeps multiple copies of data for reliability.

Examples

- Google File System (GFS)
- Hadoop Distributed File System (HDFS)
- Microsoft DFS
- CephFS

Parallel File System (PFS)

- A Parallel File System allows multiple clients or applications to read/write different parts of a file simultaneously, improving throughput and performance.



- **Key Characteristics**

- Parallel Data Access:
 - Files are striped across multiple storage servers.
- High Performance:
 - Optimized for speed, especially for large datasets.
- Scalability:
 - Can handle massive amounts of data and many concurrent users.
- Dedicated Metadata & I/O Servers:
 - Separate control and data paths for efficiency.
- Low Latency:
 - Designed for high-speed networks (InfiniBand, high-bandwidth Ethernet).

How It Works

- Files are split into blocks and stored across many disk servers.
- Clients access blocks in parallel via multiple I/O paths.
- Metadata servers track file location and permissions.
- This parallelism increases read/write speed drastically.

Examples

- Lustre
- IBM GPFS (Spectrum Scale)
- BeeGFS
- PanFS (Panasas)

Parallel vs Distributed File Systems

Feature	Parallel File System	Distributed File System
Data Access	Concurrent I/O from multiple clients	Remote/replicated access through master node
Use Case	High-Performance Computing, analytics	Cloud storage, web-scale data
Architecture	Shared data nodes, parallel I/O paths	Master-worker with replication
Examples	Lustre, GPFS	HDFS, GFS

Archive Storage

- Archive Storage is low-cost, long-term, rarely accessed storage for compliance or historical data.
- **Key Features**
 - Extremely cheap (lowest cost tier)
 - Very slow retrieval time (minutes to hours)
 - Designed for data that must be kept, but not frequently used
 - Optimized for cold storage
- **Use cases**
 - Old project data
 - Long-term backups
 - Medical records, legal documents
 - Scientific research data
 - Audit logs older than 1 year
 - Disaster recovery archives

Typical classes in cloud:

AWS

- S3 Glacier
- S3 Glacier Deep Archive

Google

- Archive Class
- Coldline Storage

Azure

- Cool tier
- Archive tier

Advantages

- Lowest cost storage
- High durability
- Ideal for compliance & legal retention
- Supports massive data volumes

Limitations

- Slow access (several hours for Deep Archive)
- Not suitable for regular reads/writes
- Retrieval costs may apply

Storage Type	Structure	Use Case	Example
Object	Flat	Media, backup	AWS S3
File	Hierarchical	File sharing	EFS
Block	Fixed blocks	DB, VM	EBS
Archive	Cold	Archival	Glacier

Elastic Storage

- Elastic Storage is cloud storage that can automatically scale up or down based on demand.
- It adjusts capacity without manual provisioning.
- **Key Characteristics:**
 - Automatically expands when data increases
 - Automatically reduces allocated capacity when data decreases
 - High availability and flexibility
 - Pay-as-you-grow pricing model
 - Enables fast deployment without hardware limitations

How It Works

- Elastic storage is enabled through:
 - **Distributed Storage Architecture**
 - Data is spread across multiple servers or nodes.
 - **Auto-Scaling Mechanisms**
 - Cloud monitors usage and allocates more or fewer resources automatically.
 - **Virtualized Storage Pools**
 - Multiple storage devices act as one large storage pool.
 - **APIs & Software-Defined Storage**
 - Automation tools dynamically manage capacity, durability, and redundancy.

Advantages

- No manual provisioning needed
- Highly scalable for unpredictable workloads
- Reduced storage cost (pay only for what you use)
- No hardware purchasing or maintenance
- Supports real-time applications and big data

Limitations

- Requires strong cloud connectivity
- Data management policies vary across providers
- Costs can increase if usage spikes unexpectedly

Use Cases

- Big data processing
- Machine learning workloads
- Media streaming platforms
- Enterprise backups and disaster recovery
- Rapid application scaling
- High-traffic websites

Examples:

- AWS Elastic File System (EFS)
- IBM Elastic Storage System
- Azure Elastic SAN

Data Redundancy

- Data Redundancy refers to the duplication of data across multiple locations to ensure durability, availability, and protection against failures.
- **Purpose:**
 - To prevent data loss and allow continuous access even during hardware failures, network outages, or disasters.

1. Local Redundancy (Within Same Data Center)

- Multiple copies stored on different disks/servers
- Protects against hardware failures
- Example: RAID, local replication

2. Zonal/Regional Redundancy

- Data copied across multiple zones (AZs)
- Protects against data center failure
- Example: AWS S3 Standard (replicated across 3 AZs)

3. Cross-Region / Geo-Redundancy

- Data replicated across geographically distant regions
- Protects against natural disasters
- Example: Azure GRS (Geo-Redundant Storage)

Advantages

- High durability
- Continuous availability
- Protection against accidental deletion
- Disaster recovery support
- Fault tolerance in distributed systems

Limitations

- Higher storage costs
- Increased sync/replication overhead
- Possible data inconsistencies in asynchronous replication

Methods & Mechanisms

- **RAID (Redundant Array of Independent Disks)**
Keeps mirrored or parity-based copies.
- **Replication**
Synchronous or asynchronous data copying across nodes.
- **Erasure Coding**
Splits data into chunks + parity blocks for efficient durability.
- **Snapshots & Versioning**
Restores previous versions if corruption occurs.
- **Backup Systems**
Long-term redundant copies in cold storage.

Use Cases

- Cloud storage services (S3, Azure Blob)
- Databases requiring high availability
- Enterprise disaster recovery
- Banking and financial systems
- Healthcare & government data retention
- Websites serving global users

Cloud database

- A cloud database is a database that runs on a cloud computing platform, providing flexible scalability, high availability, and managed operations.
- **Deployment Models:**
 - Managed Database:
 - Hosted and maintained by cloud providers.
 - Self-Managed Database:
 - User installs and manages it on virtual machines.
- Accessed through APIs, web consoles, or database clients.
- **Examples:** AWS RDS, Azure SQL, Google Cloud SQL.

- Databases are organized systems used to store, manage, and retrieve data efficiently.
- Two Major Categories:
 - Relational Databases (SQL)
 - Non-Relational Databases (NoSQL)
 - Special Type of NoSQL: Graph Databases

SQL (Structured Query Language)

To interact with relational databases, we use SQL, a powerful, standardized language that allows us to:

- Query data
- Insert new records
- Update existing data
- Delete records
- Perform complex operations such as JOINS (combining data from multiple tables)

SQL provides strong control over data and ensures accuracy, reliability, and consistency.

ACID Properties of Transactions

SQL maintains data integrity through ACID properties, ensuring reliable transaction processing.

- **Atomicity**
 - Treats all operations in a transaction as one single unit.
 - Either all changes are committed, or none at all (rollback).
- **Consistency**
 - Ensures data remains valid and follows all rules, constraints, and relationships.
 - Database moves from one consistent state to another.
- **Isolation**
 - Each transaction runs independently, as if it were the only one executing.
 - Prevents interference between concurrent transactions.
- **Durability**
 - Once a transaction is committed, its changes are permanent.
 - Data remains safe even after system crashes or failures.

Core Features

- Highly structured data
- Strong consistency
- Complex queries using JOINS
- Supports transactions
- Based on relational model (E.F. Codd)

Examples:

- MySQL
- PostgreSQL
- Oracle DB
- MS SQL Server
- MariaDB

Advantages

- Strong data integrity
- Precise schema control
- Great for transactional systems
- Mature technology with strong tooling

Limitations

- Not ideal for unstructured or rapidly changing data
- Limited horizontal scalability
- JOIN-heavy queries may slow at scale

Non-Relational Databases (NoSQL)

- NoSQL databases store data in flexible, schema-less structures, designed for scalability and handling unstructured/semi-structured data unlike traditional SQL databases.
- They do not use tables and rows.
- Instead, they organize data using flexible structures such as:
 - Document-based (JSON-like documents)
 - Key-value pairs
 - Column-family stores
 - Graph structures
- **Examples of NoSQL Databases**
 - Cassandra: A highly scalable database. It handles large data across many servers with no single point of failure.
 - CouchDB: Uses a document-oriented model. It is designed to store data in a way that scales horizontally.
 - Neo4j: A graph database. It stores data as nodes and relationships, which is great for analyzing connected data

Key Characteristics

- Flexible schema design
- Scalable across multiple servers
- Can store unstructured or semi-structured data
- High performance for large volumes of data
- Horizontal scaling (adding more machines)
- Distributed data storage
- Easily handles big data
- Supports different data models (document, key-value, columnar, graph)
- High availability with fault tolerance
- Schema-less data storage

Advantages

- Highly scalable (horizontal scaling)
- Handles unstructured data
- Flexible schema
- High performance for large workloads
- Ideal for cloud-native applications

Limitations

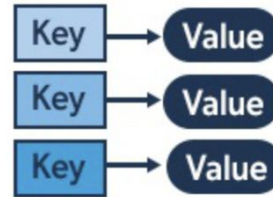
- Weaker consistency (eventual consistency) in many systems
- Limited support for multi-row transactions
- No standard query language across all NoSQL types

NoSQL – Types

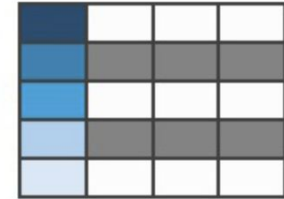
1. Key-Value Databases
2. Document Databases
3. Column Family Database
4. Graph Database

NoSQL

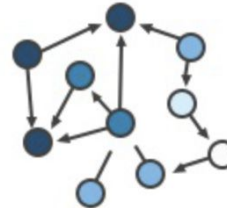
Key-Value



Column-Family



Graph



Document



1. Key-Value Databases

- Key → arbitrary value (could be string, object, blob, etc.)
- Simplest NoSQL model: data is retrieved by a unique key.
- Extremely fast lookup.
- Great for caching, storing session data, simple high-speed retrievals
- Store data as key-value pairs
- Examples: Redis, DynamoDB

Example:

"user123" → "Rita Sharma"

"cart789" → ["ItemA", "ItemB", "ItemC"]

"session567" → { "loginTime": "10:45AM", "role": "admin" }

- Keys must be known for retrieval.
- Values are **opaque**—the system does not index their contents.

2. Document Databases

- Stores data as documents rather than rows/columns.
- Schema is flexible — different documents can have different fields.
- Good for storing semi-structured or varying data;
- easier to map to objects in applications
- JSON-like documents
- Examples: MongoDB, CouchDB

Example Employee Document

```
{  
  "emp_id": 101,  
  "name": "Rita",  
  "department": "Sales",  
  "skills": ["Negotiation", "Communication"],  
  "address": { "city": "Pokhara", "ward": 10 }  
}
```

Document for another employee:

```
{  
  "emp_id": 102,  
  "name": "Sam",  
  "age": 25,  
  "projects": ["Alpha", "Delta"]  
}
```

3. Column-Family Databases

- Store data in column groups
- Great for Big Data
- Optimized for reading and writing large volumes of data;
- efficient storage/compression;
- high throughput;
- good for big data analytics, time-series, logging, large-scale distributed data.
- Examples: Cassandra, HBase

Example Table (Articles)

Row Key	Title	Author	Views
row1	"AI Basics"	"Alex"	1200
row2	"Big Data"	"Mira"	980

All values of one column are stored together, not row by row.

Column-Family: Title

row1 → "AI Basics"

row2 → "Big Data"

Column-Family: Author

row1 → "Alex"

row2 → "Mira"

Column-Family: Views

row1 → 1200

row2 → 980

4. Graph Databases

- Uses nodes + edges
- Designed for relationships
- Excellent for modeling and querying complex relationships
 - e.g. social networks, recommendation systems, connected data.
- Fast at relationship traversals and path-finding.
- Examples: Neo4j, Amazon Neptune

Graph Databases

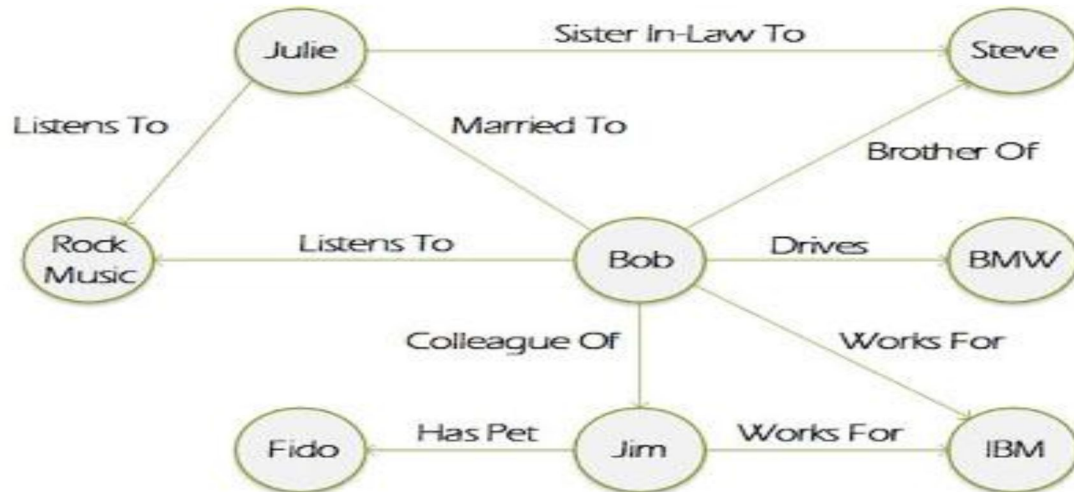
- Ideal for systems where relationships matter more than data itself.
- Graph Databases store data using a graph structure, made up of:
 - Nodes → represent entities (e.g., person, product, page)
 - Edges → represent relationships or connections between nodes
- This structure allows data to be modeled exactly how it connects in real life.
- **Examples of Graph Databases**
 - Neo4j: A popular graph database used for storing and analyzing connected data like social networks.
 - Amazon Neptune: A cloud-based database that supports graph models, ideal for handling large relationships.
 - OrientDB: A graph database that also supports document models, making it versatile for different needs.

Characteristics of Graph Databases

- Data is stored as nodes and edges
- Easily represents relationships between data
- Supports flexible data models
- Uses graph theory to manage data
- Can handle complex queries quickly
- Ideal for connected data, like social networks
- Scalable for large datasets
- Allows real-time data processing
- Provides fast searching of relationships
- Supports multi-level relationships

Main Components

- Nodes: Represent people, products, devices, accounts
- Edges: Represent relationships (friend, likes, follows, connected-to)
- Properties: Key-value attributes for both nodes and edges



Use Cases

- Social networks (friends, followers, interactions)
- Fraud detection (detect unusual connections)
- Recommendation systems
- Knowledge graphs (Google uses this)
- Network analysis
- Access control modeling

SQL vs NoSQL vs Graph DB Comparison

Feature	SQL	NoSQL	Graph
Schema	Fixed	Flexible	Relationship-based
Query	SQL	API / Query Language	Cypher / Gremlin
Scalability	Vertical	Horizontal	Depends on Model
Consistency	Strong	Eventual	Strong (within nodes)
Use Cases	Finance, ERP	IoT, Big Data	Social, AI/ML

SQL vs. NoSQL Databases in the Cloud

SQL	NoSQL
Set Up Your Development Environment	Stands for Not Only SQL
Relational database management system (RDBMS)	Non-relational database management system
Suitable for structured data with predefined schema	Suitable for unstructured and semi-structured data
Data is stored in tables with columns and rows	Data is stored in collections or documents
Follows ACID properties (Atomicity, Consistency, Isolation, Durability) for transaction management	Does not necessarily follow ACID properties
Supports JOIN and complex queries	Does not support JOIN and complex queries
Uses normalized data structure	Uses denormalized data structure
Requires vertical scaling to handle large volumes of data	Horizontal scaling is possible to handle large volumes of data
Examples: MySQL, PostgreSQL, Oracle, SQL Server, Microsoft SQL Server	Examples: MongoDB, Cassandra, Couchbase, Amazon , DynamoDB, Redis

Database Replication & Sharding

- **Replication:**
 - Duplicating data across multiple servers for availability and fault tolerance.
 - *Types:* Master-Slave, Multi-Master, Synchronous/Asynchronous.
- **Sharding:**
 - Splitting data horizontally across databases or nodes.
 - Improves scalability and performance.
 - Each shard handles a portion of the dataset.
- **Benefits:**
 - Load balancing
 - Disaster recovery
 - Reduced latency

Backup, Restore & Security

- **Backup Strategies:**
 - Point-in-time recovery
 - Automated snapshots
 - Incremental backups
- **Restore Procedures:**
 - Disaster recovery through replicas and backups
- **Security Measures:**
 - Encryption at rest: Protects stored data.
 - Encryption in transit: Secures data during transfer.
 - Access Control: IAM, role-based permissions.
 - Compliance: GDPR, HIPAA.

Real-World Case Study

- Facebook & Cassandra:
 - Handles billions of write operations daily.
 - Distributed NoSQL system offering high availability.
- LinkedIn & GraphDB:
 - Uses Apache Giraph for connection analysis.
 - Powers “People You May Know” and recommendation systems.
- Netflix & AWS RDS + DynamoDB:
 - Combines SQL for metadata and NoSQL for high-throughput content delivery.

Thank you